

eQMS - Audit trail

1. Purpose

The audit trail provides **traceability, accountability, and compliance** in accordance with GxP regulations (e.g., FDA 21 CFR Part 11, EU Annex 11). It captures **who did what, when, and to what data**.

It supports:

- Investigations of user actions.
- Reconstruction of document history.
- Demonstrating regulatory compliance during inspections.

2. When Audit Events Are Created

Audit events are generated for the following types of operations:

Operation Type	Example	Mode
Create/Update/Delete	Creating, editing, or deleting a document, tenant, user, role	Synchronous Outbox → Queue
Read / Access	Viewing document metadata, content, historical versions, lists, users	Asynchronous → Queue
Failed attempts	Unauthorized access, ABAC violations	Asynchronous → Queue
System events	Tenant creation, role assignment, permission changes	Synchronous Outbox → Queue

Note: Using **synchronous outbox for CUD actions** ensures audit events are not lost if transaction rolls back. Read operations use **asynchronous logging**, as they do not change system state.

3. Audit Event Content

Each audit event contains the following information:

```

1  {
2    "eventId": "uuid",
3    "tenantId": "tenant-123",
4    "timestamp": "2025-11-15T12:34:56Z",
5    "actor": {
6      "userId": "u-123",
7      "roles": ["Viewer"]
8    },
9    "action": "Document.Read",
10   "objectType": "Document",
11   "objectId": "doc-123",
12   "requestId": "req-456",
13   "correlationId": "corr-789",
14   "sourceIp": "4.3.2.1",
15   "requestUrl": "/api/v1/documents/doc-123/content",
16   "fieldsReturned": ["documentId", "title", "version", "classification"],
17   "payload": {
18     "before": null,
19     "after": null
20   },
21   "system": {
22     "application": "DocumentService",
23     "version": "1.0.0"
24   },
25   "status": "Success",
26   "reason": null
27 }
28

```

Field Explanation

Field	Description
<code>eventId</code>	Unique identifier for the audit event
<code>tenantId</code>	Tenant in which the action occurred
<code>timestamp</code>	UTC time of the event
<code>actor</code>	User performing the action, including roles
<code>action</code>	Type of operation (<code>Document.Read</code> , <code>Document.Update</code> , etc.)
<code>objectType</code>	Resource type (<code>Document</code> , <code>User</code> , <code>Role</code>)
<code>objectId</code>	ID of the resource involved (nullable for list/search)
<code>requestId</code>	Unique ID for the API request
<code>correlationId</code>	Optional cross-service correlation ID
<code>sourceIp</code>	IP of the caller

<code>requestUrl</code>	Full request URL including path and query parameters
<code>fieldsReturned</code>	Fields returned to user (for reads)
<code>payload</code>	Snapshot of changes for CUD operations (<code>before</code> / <code>after</code>)
<code>system</code>	Application/service information
<code>status</code>	Success, Failed, or Denied
<code>reason</code>	Optional reason for failure or denial

4. Storage Architecture

The audit trail is designed for **durability, immutability, and scalability**.

4.1 Storage Components

1. Primary Database (PostgreSQL / MS SQL)

- Stores **CUD-related audit events** via **outbox pattern**.
- Ensures audit entry is **transactional** with the data change.
- Enables **querying and reporting** of critical events.

2. Queue (Azure Service Bus / Storage Queue)

- Asynchronous read access logs are pushed here.
- Guarantees **at-least-once delivery** to the audit worker.

3. Audit Worker

- Consumes queue messages and writes to **append-only audit store**.
- Ensures decoupling from API latency.

4. Append-Only Store / Audit Database

- Optimized for **immutability** (no deletes, no updates).
- Could be a dedicated SQL table or a specialized append-only store.
- Used for **auditor queries and exports**.

5. Reliability and GxP Compliance

To ensure audit events are **not lost**:

1. Outbox Pattern (CUD operations)

- Outbox entry written in same transaction as the data change.
- Worker picks it up and persists to audit store.

- Guarantees **audit entry is not lost** if transaction rolls back.

2. Queue for Reads

- Read accesses are sent asynchronously to a queue.
- Worker retries on failure, ensuring eventual consistency.

3. Retries & Dead-letter Handling

- Queue worker implements retries for transient errors.
- Failed messages are sent to **dead-letter queue** for manual inspection.

4. Immutability

- Once written to append-only store, audit entries cannot be deleted or altered.

The **Outbox Pattern** ensures that **audit events for CUD operations (Create, Update, Delete)** are **never lost** and are **transactionally consistent** with the underlying data change.

5.1 How Outbox Works

1. Within the Same Transaction

- When an API performs a **CUD operation** (e.g., creating a document), the system:
 - Updates the main entity table (e.g., `Documents`)
 - Inserts a corresponding **audit event into the Outbox table**
- Both actions happen in **one database transaction**.

2. Transaction Commit

- If the transaction **commits**, the audit event is guaranteed to exist in the outbox.
- If the transaction **rolls back**, **neither the entity nor the audit entry is saved**, ensuring consistency.

3. Worker Processing

- A **separate background worker** continuously polls the Outbox table.
- It reads new entries and writes them to the **append-only audit store** or **queue for asynchronous processing**.

4. Marking as Processed

- Once successfully persisted to the audit store, the worker **marks the outbox entry as processed**.
- Optional cleanup or archival can be done after a retention period.

5.2 Outbox Table Structure

```
1 CREATE TABLE AuditOutbox (  
2     OutboxId UUID PRIMARY KEY,  
3     TenantId UUID NOT NULL,  
4     EventType VARCHAR(100) NOT NULL,
```

```

5      ObjectType VARCHAR(50) NOT NULL,
6      ObjectId UUID,
7      Payload JSONB NOT NULL,
8      CreatedAt TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
9      ProcessedAt TIMESTAMP WITH TIME ZONE NULL
10 );
11

```

- **ProcessedAt** → indicates the event has been handled by the worker.
- **Payload** → serialized audit event content.

5.3 Retry and Failure Handling

- Worker retries failed writes to the audit store automatically.
- Failed events can be moved to a **dead-letter table or queue** for manual intervention.
- Ensures **no audit events are lost**, satisfying GxP traceability requirements.

5.4 Outbox Lifecycle and Clearing

1. Processed Flag

- After the worker successfully writes an audit event to the append-only audit store (or queue), the outbox entry is marked as **ProcessedAt = timestamp**.
- This prevents re-processing of the same entry.

2. Retention Period

- Outbox entries are **retained for a configurable period** (e.g., 7–30 days) after being marked as processed.
- Purpose:
 - Allows **retries** in case of worker failure.
 - Facilitates **troubleshooting** if an audit event appears missing in the final store.

3. Clearing / Cleanup

- After the retention period, a **scheduled cleanup job** deletes processed outbox entries.
- Important considerations:
 - Only delete **processed entries**.
 - Keep **unprocessed entries** until successfully persisted.
 - Ensure **deletion does not violate GxP traceability**; only entries already persisted to append-only store are eligible.

4. Failed Events

- Entries that fail to be processed are **not cleared** until manually investigated or successfully processed.
- Could be moved to a **dead-letter table or queue**.

5.5 Outbox Lifecycle

Stage	Description
Insert	Outbox entry created within same transaction as CUD action
Process	Worker reads entry and writes to append-only store
Marked	<code>ProcessedAt</code> timestamp recorded
Retain	Entry kept for configurable period for retries/troubleshooting
Delete	Scheduled cleanup deletes processed entries older than retention period

6. Audit Event Lifecycle

1 | API Request → RBAC/ABAC → Action Performed → Outbox / Queue → Worker → Append-only Audit Store → Auditor Access

1. **API layer** logs events either synchronously (CUD) or asynchronously (read).
2. **Worker** ensures delivery to append-only store.
3. **Auditor or system admin** can query/export audit events, without modifying them.

7. Accessing Audit Trail

• Endpoints:

- `GET /api/v1/audit` – search by tenant, user, object, action, date range.
- `POST /api/v1/audit/export` – generate downloadable CSV/JSON export.
- `GET /api/v1/audit/export/{jobId}/file` – download export file.

• Access Control:

- Only users with **audit.read** or **audit.export** permission can access.
- Regular users **cannot query audit trail**.

- **Field-level filtering** is supported for export, e.g., only events for certain documents or actions.

8. Additional Considerations

1. High Volume

- Queues decouple API load from audit storage.
- Worker can scale horizontally to handle spikes.

2. **Historical Integrity**

- Audit entries for historical document versions indicate which version was accessed.

3. **Security**

- Audit store encrypted at rest.
- Access restricted to auditors and system administrators.

4. **Cost Optimization**

- Use SQL database for initial storage, append-only table for immutable logs.
- Queue allows batching writes to reduce transactional overhead.